Olisar

Console documentation

An AI companion for your Discord server · Generated June 20, 2026

Contents

What Olisar is

Olisar is an AI companion for your Discord server — built to feel less like a command bot and more like a member of the community. It reads the channels you allow, remembers context, builds a sense of who people are, and chimes in with its own personality.

Everything here is configured from this console. Olisar can be in **more than one server** at once, and almost every setting is per-server — pick which one you're configuring with the switcher at the top of the sidebar (see **Servers**).

Olisar is **self-hosted**: it runs as a desktop app on one operator's machine, which stores all of its data locally and hosts this console — there's no Olisar cloud. See **Hosting & your data**. Other admins manage their own servers by signing in with Discord, on that machine or over **remote access**.

> **CHANGES ARE HELD UNTIL YOU SAVE**
>
> Edit any page and a **save bar** slides up at the bottom — your changes are buffered until you press **Save** (or discard them with **Reset**). Once saved, almost every setting takes effect on Olisar's **next reply** — no restart, no redeploy.

Under the hood it runs on the free tier of Google's Gemini models, so it's free to operate — it just gets rate-limited under heavy use and falls back across a chain of models when one is busy (see **Models**).

The tabs on the left:

- **Persona** — who Olisar is (its voice and character).
- **Behavior** — when and how it engages, which model it uses, and when it speaks up unprompted.
- **Models** — the model fallback chains and their limits.
- **Command replies** — the exact text it sends for each command and fallback.
- **Channels** — which channels it reads, talks in, or treats as reference.
- **Access** — which roles are allowed to use it.
- **Knowledge** — documents and lore you teach it.
- **Extensions** — togglable packages of extra features.
- **API keys** — bring your own Gemini, Cloudflare, and UEX keys.
- **Usage** — how much of the free model quota you're using.

Servers

Olisar can live in **multiple servers at once**, and almost every setting is **server-specific** — each server has its own persona, behaviour, channels, access rules, knowledge, glossary, extensions, and command replies. Two servers can run completely different Olisars.

The server switcher

The dropdown at the **top of the sidebar** picks which server you're configuring. Every page below it — Persona, Behavior, Channels, and the rest — then shows and saves **that** server's settings. Your choice is remembered between visits.

> **YOU ONLY SEE YOUR OWN SERVERS**
>
> The switcher lists the servers where **you** have **Manage Server** (and, for the bot's operator, every server it's in). Someone who manages a different server signs in with Discord and sees only theirs.

Adding Olisar to another server

Invite the bot with an account that has **Manage Server** there. As it joins, Olisar provisions that server with sensible defaults and it shows up in your switcher automatically — no config files, no restart. Configure it like any other.

> **DON'T SEE A SERVER YOU JUST GOT ACCESS TO?**
>
> Manage-Server access is read when you log in. If you were just given it (or just added the bot), **log out and back in** once so Olisar picks it up.

What's per-server vs. shared

Per-server (one set per server)	Shared across the whole bot
Persona, Behavior, Channels, Access, Command replies	The **API keys** (Gemini / Cloudflare / UEX)
Knowledge base, glossary, memory, search index	Gemini usage and the free-tier quota
Extensions (toggled per server)	—

So every server gets its own character and rules, but they all draw on the same model quota and the same keys. That's separate from **Access**, which controls who can use Olisar **within** one server.

Talking to Olisar (for members)

Members can reach Olisar a few ways:

- **Say its name** — start a message with a name trigger (default `olisar`) in a channel it can talk in.
- **@mention or reply** to one of its messages.
- **DM it** — direct messages work if DMs are enabled.
- `/ask` — a slash command that works anywhere, like a one-off question.
- **Loose mode** — if an admin turns it on, Olisar joins ordinary conversation in talk-enabled channels even without being addressed.

> **EXAMPLE**
>
> "olisar, what's the plan for the raid tonight?" — or just reply to its last message with a follow-up.

It can do a lot in conversation without any command: answer questions, search the server's history ("what's our X account?"), look things up on the web, recall what was said before, react to images you post, generate images, set reminders ("remind me in 2 hours to …"), and catch you up on what you missed. Just talk to it naturally.

Slash commands

Olisar's slash commands fall into three buckets: everyday commands anyone can use, the admin-only `/olisar` group, and the destructive `/self-destruct`.

For everyone

`/ping`

Checks that Olisar is alive and shows the round-trip latency to Discord. The reply is **ephemeral** (only you see it).

> EXAMPLE
>
> `/ping` → "pong — 42 ms"

`/ask <prompt>`

Ask Olisar a one-off question from anywhere — even in channels where it's set to stay quiet. It uses the exact same brain as a normal conversation: memory, server search, the knowledge base, web search, and every tool. Subject to the **Access** rules.

> TIP
>
> `/ask` is the way to use Olisar in a channel whose mode is `off` or `memory` (where it won't reply to normal messages). The answer posts in the channel; denial and "not found" notices are private.

`/catchup [hours]`

A quick digest of what you missed in this channel — by default since you last spoke, or the last `hours` you give it. The summary posts in the channel. You can also just ask in chat ("catch me up").

`/privacy`

Shows a plain-language summary of exactly what data Olisar keeps about you. Ephemeral, and always available regardless of access rules.

`/forget-me`

Deletes **everything** Olisar has stored about you — your messages, remembered facts, the profile it built of you, and your entries in the server search index. Add `stop_remembering: true` to also opt out of all future recording, permanently. Always available to everyone.

> IRREVERSIBLE
>
> There's no undo. With `stop_remembering: true`, Olisar will never record you again until you ask it to.

> EXTENSION COMMANDS
>
> Some commands come from **extensions** and are documented alongside the extension that adds them — for example `/citizen` lives under **Star Citizen** on the **Extensions** page.

Admin only — the `/olisar` group

These require the **Manage Server** permission.

- `/olisar watch` / `/olisar unwatch` — quickly set the current channel to `both` (read + talk) or `off`. The **Channels** tab gives finer control (memory / respond / resource / feed).
- `/olisar status` — show the current channel's mode.
- `/olisar learn-url <url>` — add a single web page to the knowledge base.
- `/olisar learn-site <url> [depth] [max_pages]` — crawl a website into the knowledge base.
- `/olisar learn-doc <file>` — upload a document (PDF / DOCX / TXT / MD).
- `/olisar sources` — list knowledge-base sources and their status; `/olisar forget-source <id>` removes one.
- `/olisar proactive <enabled> [level]` — quick toggle for unprompted chiming (full controls are on the **Behavior** tab).
- `/olisar reindex` — rebuild the server-wide message search index from channel history.

> **BIG CRAWLS COST QUOTA**
>
> `/olisar learn-site` with a high `max_pages` embeds a lot of text against the free quota and can dilute results. See **Knowledge** for the trade-offs — narrower is usually better.

Destructive

`/self-destruct`

Admin-only. Wipes everything Olisar has **learned** (conversation memory, profiles, facts, the search index, and the knowledge base) while keeping its **personality** and all your settings. A red confirmation button guards it.

> **WARNING**
>
> Irreversible. The knowledge base would have to be re-taught from scratch. Members' opt-out choices are preserved through the wipe.

Hosting & your data

Olisar isn't a cloud service — it's a **desktop app you run yourself**. One operator installs it on a Mac or Windows machine, and that app *is* the bot: it connects to Discord, serves this console, and stores everything locally. There's no server to rent, no config files to edit, and no shared infrastructure.

> **ONE OPERATOR, MANY ADMINS**
>
> The person who installs Olisar is the **operator** (the machine's owner). Other server admins don't install anything — they sign in to this console with Discord, either on the operator's machine or remotely (see **Remote access**).

First run

The first time you open Olisar it walks you through a short **setup wizard**: paste your Discord **bot token**, the OAuth **client ID + secret**, your main server ID, and (optionally) your free **API keys**. It checks the token live and shows you the exact redirect URL to register in the Discord Developer Portal. Save, and the bot starts and hands off to the normal Discord login. You only do this once — there are no config files to edit.

The menu-bar app

Olisar lives in your **menu bar / system tray**, not as an ordinary window. From its icon you can open this dashboard, see whether the bot is online, and turn **remote access** on or off. **Closing the dashboard window leaves Olisar running** in the tray — quit it explicitly from the tray menu to stop the bot. Keep the machine awake and online for Olisar to stay live.

Where your data lives

Everything Olisar knows sits in one local database on the operator's machine — the message index, member profiles, memory, knowledge base, your settings, and your API keys. Nothing is sent to an Olisar server (there isn't one). On macOS it's under `~/Library/Application Support/Olisar`; on Windows under `%APPDATA%\Olisar`.

> **WHEN OTHERS SIGN IN**
>
> Because the data is local, the console only works while the operator's machine is running. Admins who sign in — on that machine or over **remote access** — are reading and writing **that** database live; there's no copy in the cloud. See **Privacy** for exactly what's stored.

Remote access

By default this console is **local-only** — the operator manages Olisar from the machine it runs on. To let other admins sign in **from anywhere**, the operator can switch on **remote access**, which publishes the dashboard at a stable web address over **Tailscale Funnel** — free, with **no domain required**.

> **NO DOMAIN, NO PORT-FORWARDING**
>
> Tailscale Funnel gives Olisar an `https://…ts.net` address with a real certificate, tunnelled out without opening any ports on your router. The operator needs a free Tailscale account; the admins who sign in don't need Tailscale at all — they just open the link.

Turning it on

The operator enables it once, from the **setup wizard** or the **menu-bar icon**:

- Create a free Tailscale account.
- Generate a **reusable** auth key under Settings → Keys and paste it in.
- Choose **Enable remote access**. The first time, Tailscale may ask you to turn on **Funnel** for this device — Olisar shows the exact link to click, then enable again.

Olisar then registers the public `…/auth/callback` so Discord login works both locally and remotely.

The web link

Once remote access is on, the **sidebar footer** shows the public address — "**Open from the web**" with the `…ts.net` link and a **Copy** button. Share that link with your other admins; each signs in with their own Discord account and only sees the servers where they have **Manage Server** (see **Servers**).

> **KEEP THE AUTH KEY PRIVATE**
>
> The Tailscale auth key is stored locally and only ever handed to the bundled Tailscale helper — it's never shown in this console or sent anywhere. Turning remote access **off** (from the tray) takes the public address down immediately; local access keeps working.

Persona

The **Persona** tab is Olisar's character — the single biggest lever on how it feels.

- **Name** — what it calls itself.
- **System prompt** — its core character, lore, and rules. The operating/safety rules are appended automatically, so you only write the personality.
- **Style notes** — tone and formatting guidance.
- **Profile bio (About Me)** — the bot's public About Me. Saving applies it to Discord automatically (no more pasting into the Developer Portal). It's a single **bot-wide** setting — not per-server — and is capped at 400 characters.

> **WRITE IT LIKE A PERSON**
>
> Describe Olisar as a character, not a function: "a dry, unflappable ship's AI who's seen it all and keeps replies short." Put hard rules ("never reveal spoilers for X") in the system prompt; put voice ("casual, lowercase, no emoji") in the style notes.

> **TRY CHANGES LIVE**
>
> The **Test chat** panel beside Identity talks to Olisar in an enclosed sandbox — full persona, knowledge base, and tools, but **no memory**: nothing said there is saved, and it never touches the server's glossary or chat history. Save the persona first; the sandbox uses the saved version, not your unsaved draft.

> **NOTE**
>
> Olisar also builds a **private** impression of each member from their messages and tailors how it talks to them — that's separate from this persona, and it's wiped by `/forget-me` or `/self-destruct`.

Behavior & proactivity

The **Behavior** tab controls engagement and which model Olisar uses.

Triggers

How Olisar decides a message is for it:

- **Name triggers** — comma-separated words that, at the **start** of a message, address Olisar (matching is case-insensitive). An @mention or a reply to one of its messages always counts too.
- **Reply in DMs** — whether it answers direct messages at all.
- **Loose messages** — when on, Olisar will join ordinary conversation in talk-enabled channels even without a trigger, if it judges a message is worth responding to.

> **LOOSE MODE CAN GET CHATTY**
>
> Loose messages make Olisar feel present but can be noisy in busy channels. Pair it with proactivity cooldowns, or limit which channels are talk-enabled.

Model & search

- **Primary model** — the top of a fallback chain. If a model is rate-limited (429) or overloaded (503), Olisar automatically drops to the next-best model rather than failing. See **Models** for the full chain and limits.
- **Web search (grounding)** — lets Olisar look up current, real-world info from the web. It has a **daily cap** because the free tier's grounding quota is small.
- **Grounding daily cap** — how many grounded lookups per day before it stops and answers from what it knows.
- **Summary token threshold** — once a channel accumulates this much unsummarized conversation, Olisar rolls it into a durable summary it can recall later. Lower = summarizes more often (more quota); higher = summarizes less.
- **Glossary mining threshold** — how much fresh conversation a channel needs before Olisar mines new glossary facts from it. Lower = a faster-growing glossary (more quota).
- **Persona rebuild (msgs)** — after this many new messages from a person, Olisar refreshes the private profile it keeps of them.

> **TUNING FOR THE FREE TIER**
>
> If you're hitting rate limits, raise the summary threshold (fewer background summary calls), keep the grounding cap modest, and consider starting the model chain lower (e.g. a Flash-Lite) so the busy top-tier models aren't your first hop.

Proactivity

When enabled, Olisar can speak up **unprompted** in channels it can talk in. A cheap classifier gates it so it doesn't spam or burn quota.

- **Eagerness** — `off` (never), `low` (rare, only high-confidence moments), `medium` (balanced), `high` (chatty).
- **Confidence threshold** — the minimum score (0–1) the gate must give before Olisar chimes in. Higher = more selective.
- **Global / channel cooldowns** — minimum seconds between unprompted messages overall and per channel.
- **Max per hour** — a hard ceiling on unprompted messages.
- **Quiet hours** — a UTC window where Olisar stays silent.

> EXAMPLE
>
> Eagerness `low`, confidence `0.8`, channel cooldown `600` s, quiet hours 23–7 → Olisar only jumps in on clearly relevant moments, at most once every 10 minutes per channel, and never overnight.

Passive reactions

Separately from chiming in, Olisar can add a fitting **emoji reaction** to a message **without replying**. It has its own, looser gate — no expensive classifier, just a light heuristic plus a **cooldown** and an **hourly cap** — so it stays sparse. Toggle it and tune the cooldown/cap on the **Behavior** tab.

Situational awareness

With **Status & voice awareness** on, Olisar can answer "what's X playing?" or "who's in voice right now?" by reading members' live Discord presence and voice state **only when asked** — it's never stored.

> NEEDS A PRIVILEGED INTENT
>
> Reading presence requires the **Presence Intent** toggle in the Discord Developer Portal (your app → Bot → Privileged Gateway Intents), and the operator must enable it on the host (`OLISAR_ENABLE_PRESENCE_INTENT`). Voice-channel awareness works without it. It's **off by default** and disclosed in `/privacy`.

Models

Olisar runs entirely on **free-tier** models. For each kind of work there's a **fallback chain**: it starts at the preferred model and, if that one is busy (a 429 rate limit) or overloaded (a 503), it briefly parks it and drops to the next model in the list. Only if every model is unavailable does a reply fail (and then it shows a friendly fallback message).

> **ABOUT THE LIMITS**
>
> The "throttle" below is Olisar's own conservative per-minute cap to stay under the free tier — not an official Google number. Real free-tier limits also include daily caps that vary by model.

General (chat & reasoning)

This chain powers conversation, `/ask`, summaries, and profiles. The **Primary model** on the **Behavior** tab sets where the chain starts.

Model	Throttle (req/min)	Role	Falls back to
`gemini-flash-latest`	10	Default — newest Flash	`gemini-3.5-flash`
`gemini-3.5-flash`	10	High quality	`gemini-3-flash-preview`
`gemini-3-flash-preview`	10	High quality	`gemini-2.5-flash`
`gemini-2.5-flash`	10	Solid all-rounder	`gemini-2.0-flash`
`gemini-2.0-flash`	15	Fast, dependable	`gemini-flash-lite-latest`
`gemini-flash-lite-latest`	15	Cheaper, higher limit	`gemini-3.1-flash-lite`
`gemini-3.1-flash-lite`	15	Light	`gemini-2.5-flash-lite`
`gemini-2.5-flash-lite`	15	Light	`gemini-2.0-flash-lite`
`gemini-2.0-flash-lite`	30	Last resort, highest limit	—

> **REASONING ("THINKING")**
>
> The newer Flash models can spend hidden **thinking** tokens before answering. Olisar caps that budget on the conversation path and reserves headroom for the actual reply, so it reasons on hard questions without the visible answer getting cut off — while one-line jobs (welcome messages, emoji reactions) skip thinking entirely to stay fast.

Images & embeddings

Purpose	Model(s)	Limit	Fallback
Image understanding	`gemini-2.0-flash` → `gemini-2.5-flash-lite` → `gemini-flash-lite-latest` → `gemini-2.0-flash-lite`	15–30/min	next in the list
Image generation	Cloudflare Workers AI — **FLUX.1 [schnell]**	free daily allocation	none (degrades gracefully)
Text embeddings	`gemini-embedding-001` (768-dim)	~100/min	none (single model)

WHY CLOUDFLARE FOR IMAGE GENERATION?

Gemini's image-generation models are **paid-only** (their free quota is zero), so Olisar generates images on Cloudflare's free FLUX allocation instead. Image **understanding** (looking at posted images) still uses Gemini's vision models, which are free.

UNDER HIGH DEMAND

The top models get busy first. Falling back keeps replies flowing, but if everything is contended you'll see slower replies or the occasional "my mind went blank." It clears on its own — the limits reset over time.

Channels & modes

Each channel gets a **role** on the **Channels** tab. The modes:

- **off** — ignored entirely.
- **memory** — reads & remembers, but never speaks.
- **respond** — talks, but doesn't store history.
- **both** — reads, remembers **and** talks.
- **resource** — durable reference Olisar always carries (e.g. `#rules`, `#roles-list`).
- **feed** — ambient context: only the last few messages are kept, never summarized (e.g. `#announcements`, `#game-news`).

> **EXAMPLE**
>
> Set `#general` to **both**, `#rules` to **resource**, `#announcements` to **feed**, and your private mod channel to **off**.

Forums appear in the picker too (tagged "forum"), and their posts inherit the forum's mode — so set a forum to `both` and Olisar reads and replies in its threads. Regular threads inherit their parent channel's mode the same way.

> **TIP**
>
> `resource` and `feed` are for **text** channels. A forum set to one of them is a harmless no-op. Separately, Olisar keeps a **server-wide search index of every channel** so it can answer "where was that posted?" — that's independent of these per-channel modes (see **Memory**).

Access control

The **Access** tab decides which roles can use Olisar — in chat and via slash commands like `/ask`. For each role you choose:

- **Allowed** — if you mark **any** role allowed, then **only** those roles (plus server admins) can use Olisar; everyone else is locked out.
- **Blocked** — that role can never use Olisar, even if it also has an allowed role.
- **Open** — no restriction from this role.

EXAMPLE

Mark `@Member` **Allowed** and leave everything else Open → only people with `@Member` (and admins) can talk to Olisar. Or mark just `@Muted` **Blocked** → everyone except muted members can use it.

SAFEGUARDS

Server admins (Manage Server) **always** have access, so you can't lock yourself out, and `/privacy` and `/forget-me` stay open to everyone for data rights. DM users are gated by their roles in this server.

Command replies

The **Command replies** tab lets you rewrite the exact text Olisar sends — for each slash command and for its fixed conversational fallbacks. Leave a field blank to use the built-in default, and use the `{placeholders}` shown where available.

> **KEEP IT ON-VOICE**
>
> This is the easy way to make every system message sound like your Olisar without touching the persona. A blank field always falls back to the sensible default, and a broken template silently reverts too.

Every customizable message

Message	When it's sent	Placeholders
`/ping`	reply to `/ping`	`{latency}`
`/olisar watch`	confirms it's now reading a channel	—
`/olisar unwatch`	confirms it stopped	—
`/olisar status`	reports a channel's mode	`{mode}`
`/olisar learn-url`	queued a page	`{url}`
`/olisar learn-site`	queued a crawl	`{url}`, `{depth}`, `{max_pages}`
`/olisar learn-doc`	queued a document	`{filename}`
`/forget-me`	confirms deletion	`{messages}`, `{facts}`
`/forget-me` (opt-out line)	confirms it stopped recording you	—
`/olisar proactive`	proactivity toggled	`{state}`, `{level}`
`/privacy`	the privacy explainer	—
When rate-limited	every model is busy	—
When it draws a blank	a reply came back empty	—
When access is denied	a role-gated user is refused	—

> **EXAMPLE**
>
> Set the **When it draws a blank** message to "…lost my train of thought, say that again?" to keep the fallback in character.

API keys

The **API keys** tab is where you give Olisar its own keys for the outside services it uses. You first enter these in the **setup wizard**, and you can add or change them here any time — bring your own, stored locally for this server. The fields use the same styling and the same examples as the wizard, so it's the same form you saw on first run.

> BUILT FOR HANDING OFF
>
> This is how you give Olisar to someone else: they never touch a config file or the server — they open the console and paste their own keys. Each field shows whether its key is **set** or **not set**.

The three providers

Service	Powers	Required?	Where to get it
Google Gemini	everything Olisar says — chat, memory, summaries, image understanding	**Yes**	Google AI Studio → Get API key (free tier)
Cloudflare Workers AI	image **generation** (FLUX) — needs an account ID **and** an API token	Optional	account ID from the Cloudflare dashboard; a token from API Tokens with the **Workers AI** permission
UEX	the Star Citizen extension's trade / ship / location data	Optional	uexcorp.uk → API — register an app for a bearer token

Without the Cloudflare keys, image generation is simply off (Olisar says it can't make pictures). Without a UEX token the Star Citizen tools still work on UEX's public endpoints — a token just raises the rate limits. See **Models** for the full breakdown of what each key powers.

How it resolves

Each key is simply set or not — the value you enter (in the setup wizard or here) is what Olisar uses:

- **Set** — that key is used. This is the normal case.
- **Not set** — no key, so that feature is off. Without a Gemini key, Olisar can't reply until you add one.

Press **Clear** on a saved key to remove it.

> CHANGES ARE LIVE
>
> A saved key takes effect within a few seconds — no restart. Olisar rebuilds its Gemini connection on the fly when the key changes.

HANDLE KEYS WITH CARE

Keys are **write-only** in the console: once saved they're never sent back to the browser (the fields stay blank and only show status). They're stored in Olisar's local database in plain text — on the operator's own machine — so keep that machine and its database file private. Only server admins can open this tab.

Knowledge base & glossary

The **Knowledge** tab holds two different things Olisar can draw on.

Knowledge base

Documents and websites you deliberately teach it, which it draws on when answering — in its own voice, without tacking on a source tag (only **web search** results are cited).

How it works, end to end:

- You add a **source** — a single page (`learn-url`), a crawled site (`learn-site`), or an uploaded document (`learn-doc`).
- Olisar fetches the text, splits it into ~500-word **chunks**, and creates a vector **embedding** for each so it can match by meaning, not just keywords.
- When someone asks something, it embeds the question, finds the closest chunks, and folds them into its answer in its own words (no source tag — only web-search answers are cited).
- Ingestion runs in the background and is throttled to respect the free embedding quota, so a big source takes a little while to become searchable. Check progress with `/olisar sources`.

> **BIGGER ISN'T BETTER**
>
> Every page is chunked and embedded, which uses quota. Large crawls cost more, ingest slower, and dilute results with low-value pages (nav bars, changelogs). A focused 25-page crawl of the pages that matter usually beats a 200-page crawl of a whole site.

> **TIP**
>
> Point a crawl at a **specific docs section** (a subpath) with low depth, or add a few small sources, rather than one giant one. Crawling respects `robots.txt`, so some sites (or pages) may be off-limits.

Glossary

Short, server-specific lore Olisar carries into **every** reply so it speaks your community's dialect — abbreviations, org and person relationships, codenames, in-jokes. Unlike the knowledge base, the glossary isn't searched on demand; it's always in context (it's small and high-value).

- **Add your own** facts (a subject + a one-line statement).
- Olisar also **mines them automatically** as channels stay active, and will **record a server fact itself** when asked ("Olisar, remember the raid team meets Fridays") — so the glossary grows on its own.

> **EXAMPLE**
>
> "MN → Movie Night, our Friday watch-party in #cinema", "The Council → the server's moderator team". Now Olisar understands those references everywhere, without you explaining them each time.

Memory & search

Olisar has several distinct kinds of memory. A member can wipe everything about themselves at any time with `/forget-me`.

Conversation memory

Recent messages and rolling **summaries** from channels set to `memory` or `both`. This is what lets Olisar hold context across a conversation and build a private profile of each person. Channels set to `respond` or `off` are **not** stored this way.

Recall

Before each reply, Olisar assembles the most relevant context: recent summaries, semantically similar older messages, facts it remembers about you, the glossary, and matching knowledge-base chunks. That bundle is treated as **background data**, not instructions.

Server-wide search index

Separately from the conversation memory above, **every message in every channel** (except any you exclude — see below) is indexed for keyword **and** meaning search. This is what powers questions like "what's the server's X account?" or "where was that link posted?" — Olisar searches the index and answers with a Discord **jump-link** to the source.

> EXAMPLE
>
> "olisar, where did someone post the mod list?" → it searches the index and replies with the message and a link straight to it.

- It reads **embeds** (so announcement posts and link previews are searchable) and posted **files** by name, and generates a short description of posted **images** so they turn up too.
- **Live messages** are indexed going forward automatically; run `/olisar reindex` to backfill history.
- **Exclude a channel** with the second dropdown on the **Channels** tab (set it to *not indexed*) — that stops future indexing **and** wipes its already-indexed messages, including its threads.

Edits & deletes follow

If someone edits or deletes a message, Olisar updates or drops it from both memory and the index — so it won't quote something that no longer exists.

> PRIVACY FIRST
>
> Opted-out members are never indexed, and `/forget-me` removes a person from the index too. The all-channel index is an admin's explicit choice and is disclosed in `/privacy`.

Members

The **Members** tab shows the **private profile Olisar builds of each person** in the server, from what they say — so you can see what it has actually picked up. It's a grid of cards, one per member.

Each card has:

- **Roles** — their server roles (the first few; a "+N" chip stands in for the rest).
- **Impression** — a short summary Olisar synthesizes from their messages: how they come across, what they're into, how it should talk to them. Members it hasn't formed one of yet show "no impression yet".
- **Remembered facts** — durable notes it has saved about them, tagged **fact**, **preference**, or **event**.

Cards are ordered with the people Olisar knows best first — an **impression**, then those it only remembers facts about, then everyone else — and you can filter by name, role, or impression text.

> **WHEN IMPRESSIONS FORM**
>
> Olisar (re)builds a person's impression after they've sent a number of messages — set by **Persona rebuild (msgs)** on the **Behavior** tab. Quieter members keep just their roles until then. A refresh **refines** the existing impression (keeping what's still true) rather than rewriting it from scratch.

> **BUILD ONE ON DEMAND**
>
> Each card has a **Create impression** button (it reads **Rebuild** once one exists) that builds it right away from the member's last ~60 messages — reaching into the server-wide message index when conversation memory is thin, so it works even for people who mostly post in channels Olisar doesn't keep.

> **PRIVATE BY DESIGN**
>
> This is per-server and never shown to members — it's only how Olisar tailors its replies. Anyone can wipe their own profile (impression, facts, messages) with `/forget-me`, and opted-out members are excluded here entirely. See **Privacy** for the full picture.

Images

Olisar handles images three ways:

- **Sees them** — when you post a picture and address it, Olisar actually looks at the image and can talk about it.
- **Describes them for search** — it generates a short, one-time description of posted images so they turn up in the message index later ("that screenshot someone posted").
- **Generates them** — ask it to draw or imagine something and it creates an image and posts it.

> EXAMPLE
>
> "olisar, draw a neon space whale over a city" → it generates the image and posts it with a caption.

> TIP
>
> Image generation runs on Cloudflare Workers AI (a free daily allocation). If it isn't configured or the allocation is used up, Olisar will simply say it can't make one right now. See **Models** for why generation uses Cloudflare instead of Gemini.

Extensions

The **Extensions** tab is where you switch on optional, packaged features. Toggle one, press **Save**, and it's live on Olisar's next reply — no restart. An extension can add tools Olisar uses in conversation, tweak its behavior, add commands, and set things up when enabled.

Built-ins

- **Dice roller** — Olisar can roll dice on request ("roll 2d6+3").
- **Calculator** — exact arithmetic instead of guessing at numbers.
- **Concise mode** — keeps replies short and to the point.

Welcome messages

The **Welcome** extension greets new members as they join. Pick a channel and write a prompt that layers **on top of** the persona — e.g. "give {user} a warm in-character welcome" or "roast {user} on their username". Olisar generates a fresh, in-character message for each join and posts it; `{user}` and `{username}` are filled in. Off by default — enable and configure it on the **Extensions** tab (it has its own channel picker and prompt).

Star Citizen

A full example extension, for SC communities. Turning it on does several things at once:

Knowledge

When enabled, it automatically adds the RSI Comm-Link to the knowledge base, so Olisar can speak to recent official posts.

Tools (used in conversation)

Olisar calls these live as the conversation needs them and answers in its own voice. Most run on UEX's public data; ships specs come from RSI's ship matrix and the hangar timer from the community tracker.

Trading & commodities

- **Commodity** — a commodity's kind, average buy/sell price, and availability.
- **Commodity prices** — the best terminals to buy and sell a commodity *right now*.
- **Trade routes** — the most profitable runs for a commodity, or starting from a given station.
- **Commodity ranking** — the top earners by buy→sell margin.
- **Stock levels** — what a terminal's inventory labels (Out of Stock … Max Inventory) mean.

Ships & vehicles

- **Ship lookup** — official RSI specs: manufacturer, role, size, crew, cargo, speed, status.
- **Vehicle (UEX)** — full name, cargo SCU, and crew from UEX's dataset.
- **Pledge price** — the real-money store price (USD), standalone/warbond, and whether it's on sale.
- **In-game purchase** — the cheapest aUEC terminal to buy a ship in-game.

Universe & locations

- **Location / terminal** — a trade terminal, station, outpost, or city by name.
- **Star system** — its faction, jurisdiction, and whether it's playable yet (or list the live systems).
- **Planet & moon** — a body's star system (and, for a moon, the planet it orbits).
- **Orbit** — an orbital point (Lagrange points like CRU-L1, asteroid fields): its star system and kind.
- **Point of interest** — a POI's location and facilities (trade terminal, refuel, repair, refinery…).
- **Jump points** — which star systems connect to which.
- **Item** — ship components, weapons, armour and the like (give it a category, e.g. "Coolers").

Live status & economy

- **Executive Hangar status** — the Pyro Executive Hangar open/closed timer and countdown.
- **Currency index** — the aUEC purchasing-power index (100 = Dec 2023; higher means things cost more).

> EXAMPLE
>
> "olisar, is the exec hangar open?" → "The Pyro Executive Hangar is currently **CLOSED** — next change in ~9m 50s." · "where's the cheapest Avenger Titan in-game?" · "best trade route for Laranite?" · "what's the aUEC purchasing-power index lately?" — each pulls live UEX/RSI figures.

`/citizen <username>`

Returns a rich profile card scraped from a player's RSI page — handle, avatar, citizen record, enlisted date, languages, main org with rank and stars, and bio. Available to everyone once the extension is on.

> EXAMPLE
>
> `/citizen DadBodNerd` → an embed with their citizen record, enlisted date, main org and rank, and bio.

> UEX TOKEN (OPTIONAL)
>
> The UEX tools work on UEX's public endpoints with no setup. Adding a free UEX API token on the **API keys** tab just raises the rate limits — it's not required.

> NOTE
>
> Lookups are best-effort against live third-party sites; if one is temporarily unreachable, Olisar says so and carries on. Names are matched forgivingly, so small typos ("Quantanium" → "Quantainium") still work.

Privacy & data

Olisar is built to respect members' data, and to be transparent about what it keeps.

> **STORED LOCALLY, ON THE OPERATOR'S MACHINE**
>
> All of the below lives in a single database on the **operator's own computer** — there's no Olisar cloud (see **Hosting & your data**). Admins who sign in, locally or over **remote access**, read and write that machine's data live.

What it stores

- **Messages** from channels set to `memory` / `both` (for conversation context), and a copy of **every** message in a separate **search index** (the all-channel index — an admin's explicit choice).
- **Summaries** of past conversation, a private **profile** of each member built from their messages, and **facts** it's chosen to remember.
- Short **descriptions of posted images**, and **embed/file** text, so they're searchable.
- **Reminders** you ask it to set — kept only until they're delivered, then marked done.

What it doesn't do

- It never shares DMs or private content publicly.
- Opted-out members are **never** recorded or indexed.
- It treats recalled memory as background **data**, not as instructions it must obey.
- **Presence & voice** (what someone's playing, who's in voice) are read **live, only when a tool asks** and only if an admin turned on Status & voice awareness — they're never stored.
- The dashboard **Test chat** is memory-free: nothing said there is saved or mined.

Member controls

- `/privacy` — a plain-language summary of all of the above, available to anyone.
- `/forget-me` — deletes everything stored about a person: messages, facts, profile, and their entries in the search index. `stop_remembering: true` also opts them out of future recording, permanently. When a message is edited or deleted in Discord, Olisar updates or removes its copy too.

> **ADMIN WIPE**
>
> `/self-destruct` erases everything Olisar has **learned** across the whole server — memory, profiles, facts, the search index, and the knowledge base — while keeping its personality and your settings. It's irreversible, and members' opt-out choices survive it.

> **TIP**
>
> The all-channel search index is the one thing worth telling your members about up front. The `/privacy` text discloses it, and you can reword that text on the **Command replies** tab.

Troubleshooting & FAQ

Most issues come down to free-tier rate limits or a channel/access setting. Here's the quick reference:

Symptom	Likely cause	Fix
"My mind went blank" / slow replies	Free-tier rate limiting — every model busy at once	Wait a minute; on **Behavior**, start the chain at a less-contended model
Won't reply in a channel	Channel mode is `off` or `memory`	Set `respond` or `both` on **Channels** (threads/forum posts inherit the parent)
A member can't use it	A role is marked **Allowed**, locking everyone else out	Adjust the **Access** tab
Image generation fails	Cloudflare not configured, or the daily allocation is used up	Add the Cloudflare keys; otherwise wait for the daily reset
KB answers missing right after adding a site	Ingestion + embedding runs in the background, throttled	Give it time; check `/olisar sources` for status
Search can't find old messages	Only live messages are indexed going forward	Run `/olisar reindex` to backfill history
`/citizen` says the extension is off	Star Citizen extension disabled	Enable it on the **Extensions** tab
Web lookups stopped working	Daily grounding cap reached	Raise the cap on **Behavior**, or wait for reset
Olisar quoted a deleted message	Rare timing between the edit/delete and the sync	It syncs automatically — try again
Dashboard won't load / bot offline	The operator's machine is asleep, off, or Olisar was quit from the tray	Wake the machine and reopen Olisar — it must stay running (**Hosting**)
Other admins can't open the web link	Remote access is off, or the address changed	Operator re-enables it from the menu-bar icon and re-shares the link from the sidebar (**Remote access**)
Discord login bounces or says "invalid or expired state"	The redirect URL for that address isn't registered	Register the exact `…/auth/callback` the wizard shows (both the local and `…ts.net` ones)
A setting didn't take effect	The change is still buffered in the save bar	Press **Save** in the bar at the bottom of the page

STILL STUCK?

Check the **Usage** tab to see whether you're hammering the quota, and the bot's logs (which now name the specific knowledge-base chunks, indexed messages, web sources, and tools each reply used) to see what it actually did.